

EduGenie – Google Gemini Powered Learning Assistant

Project Documentation | Version 1.0

1. Project Overview

EduGenie is a lightweight AI-powered educational assistant built with a FastAPI backend, HTML/CSS/JavaScript frontend, and Google Gemini API integration. It provides question answering, concept explanations, quiz generation, text summarization, and personalized learning-path recommendations.

2. Objectives

- Build the complete EduGenie application from scratch.
- Provide a simple browser-based educational AI interface.
- Integrate Google Gemini through the Google GenAI Python SDK.
- Expose separate FastAPI endpoints for each learning feature.
- Validate user input and AI-generated quiz JSON.
- Run and test the application locally using Uvicorn.

3. Technology Stack

Python 3.14.7 – application development

FastAPI – backend API

Uvicorn – local ASGI server

Jinja2 – HTML template rendering

HTML/CSS/JavaScript – frontend

Google GenAI SDK – Gemini API integration

Pydantic – request/response validation

python-dotenv – environment configuration

Transformers + PyTorch – optional local explanation model support

4. Project Structure

```
EduGenie-AI/  
├── main.py  
├── config.py  
├── gemini_client.py  
├── schemas.py  
├── utils.py  
├── qna.py  
└── explanation_module.py
```

```
|— quiz_module.py
|— summary_module.py
|— learning_path.py
|— requirements.txt
|— .env
|— .gitignore
|— templates/
|   |— index.html
|— static/
|   |— app.js
|   |— style.css
```

5. Backend Implementation

main.py creates the FastAPI application, serves the frontend, mounts static files, provides /health, and exposes:

- /qa – educational question answering
- /explain – concept explanation
- /quiz – multiple-choice quiz generation
- /summarize – text summarization
- /learn/recommendations – personalized learning path

gemini_client.py centralizes Gemini client creation and text generation. The API key and model are read from .env.

utils.py validates input length and cleans/parses AI JSON responses.

quiz_module.py validates that generated quizzes contain the requested number of questions, exactly four options per question, a valid correct answer, and an explanation.

6. Frontend Implementation

The frontend is a responsive single-page interface. Users select a task, enter a question or topic, and click Generate. JavaScript sends the request to the matching FastAPI endpoint and displays the result.

Available tasks:

- Q&A
- Explain Concept
- Generate Quiz
- Summarize
- Recommend Learning Path

Quiz results are displayed as question cards with options, the correct answer, and an explanation. A Clear button resets the page.

7. Development and Troubleshooting Completed

- Created the complete project and Python virtual environment.
- Used CMD because PowerShell activation was blocked by execution policy.
- Installed the required dependencies.
- Configured Gemini API access through .env.
- Fixed the initial homepage Internal Server Error.
- Updated Jinja2 TemplateResponse usage for the installed environment.
- Disabled Jinja2 template caching to avoid a local Python 3.14 cache issue.
- Verified templates and static assets.
- Tested Q&A successfully.
- Diagnosed Explain Concept failures using the backend terminal.
- Confirmed that the 503 errors were Gemini model high-demand responses, not a local network problem.
- Checked connectivity to the Gemini API host.
- Changed the Gemini model configuration to a working model.

8. Functional Testing Results

Q&A – Test: “What is the largest ocean in the world?” – PASSED

Explain Concept – Test: “Photosynthesis” – PASSED

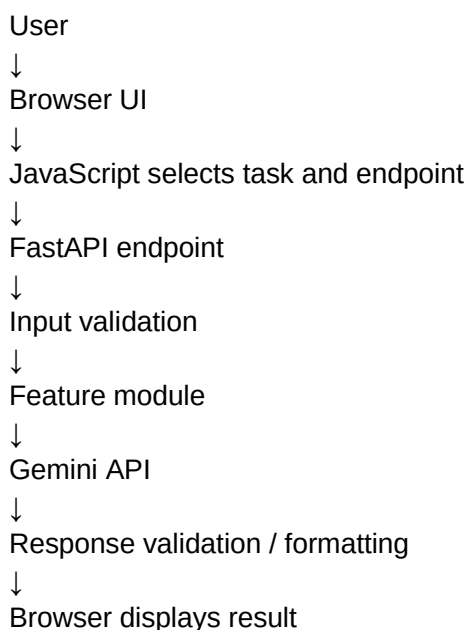
Generate Quiz – Test: “Pythagoras theorem” – PASSED

Summarize – Test: photosynthesis paragraph – PASSED

Learning Path – Test: “I want to learn SQL” – PASSED

These were functional test inputs only. Users can enter other suitable questions, topics, and text.

9. Application Flow



10. How to Run the Project

From the main EduGenie-AI folder:

```
.venv\Scripts\activate  
uvicorn main:app --reload
```

Then open:

<http://127.0.0.1:8000>

The terminal should show “Application startup complete.” when the server is ready.

11. Environment Variables

```
GEMINI_API_KEY=<your Gemini API key>  
GEMINI_MODEL=gemini-3.5-flash-lite  
EXPLANATION_PROVIDER=gemini  
LOCAL_EXPLANATION_MODEL=MBZUAI/LaMini-Flan-T5-783M  
APP_NAME=EduGenie  
MAX_INPUT_CHARS=30000  
HOST=127.0.0.1  
PORT=8000
```

The actual API key is intentionally not included in this documentation. .env is also excluded by .gitignore.

12. Future Improvements

- User progress tracking
- Multilingual learning support
- Voice input/output
- Image and PDF learning support
- Authentication and learner profiles
- Interactive quizzes where users answer before seeing solutions
- Cloud deployment
- Automated unit and API tests
- Periodic review of Gemini model and SDK recommendations

13. Conclusion

EduGenie has been implemented as a functional local AI learning assistant. The backend, frontend, Gemini integration, validation, configuration, and five core learning features were completed and tested successfully. The project is ready for further polishing, version control, automated testing, and deployment.

14. Source Documentation

The implementation follows the supplied EduGenie project documentation, which described the educational use cases, FastAPI architecture, Gemini integration, quiz structure, learning paths, frontend requirements, and local run instructions.